

StonkBench: Unified Benchmark for Synthetic Market Generation for Historical Financial Time Series

Albert Ho*

Eddison Pham*

uyenlam.ho@mail.utoronto.ca

eddisson.pham@mail.utoronto.ca

University of Toronto

Toronto, Ontario, Canada

Abstract

Synthetic data generation has become increasingly important in financial applications, particularly with generating market data such as financial securities prices. With advancements in generative architectures in text, image, and audio domains, practitioners and researchers in financial markets have shown growing interest in applying these techniques to generate market data like time series of equity prices for research and robust trading strategies backtesting. However, the current literature in this domain lacks a unified benchmark for evaluating the quality of synthetic market data for fair and reliable comparisons across different families of models. To address this gap, we introduce StonkBench, a unified benchmark for synthetic multivariate time series generation of daily equity returns through a comprehensive and standardized evaluation framework that balances statistical properties, domain-specific stylized facts observed in market data and practical downstream tasks practitioners care about, such as portfolio hedging, option price delta hedging, and alpha generation.

Keywords

Synthetic Data, Market Generator, Financial Time Series, Benchmarking

ACM Reference Format:

Albert Ho and Eddison Pham. 2025. StonkBench: Unified Benchmark for Synthetic Market Generation for Historical Financial Time Series. In . ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnn>

1 Introduction

Synthetic market data generation has emerged as a crucial tool in financial markets, particularly time series of financial asset returns, trading volume, and limit order books [12, 17], emerged from the rising popularity of generative models in other domains over the past decade. The ability to generate high-quality synthetic financial data addresses several critical challenges in the field, like simulating fat-tail behaviors and volatility clustering observed in financial

returns time series and developing back-testing procedures for prediction and (deep) hedging algorithms [20].

Despite the growing importance of market generators, there is a notable lack of standardization in evaluating the quality and effectiveness of these generative models; *Stenger et al.* surveyed 56 research papers on time series data and concluded that there are no consensus in the evaluation metrics and their interpretations. [20]. Current evaluation methods vary widely across studies, making it difficult to compare different approaches objectively and determine their relative strengths and weaknesses.

1.1 A brief overview on market generators

Synthetic market data are (multivariate) time series related to financial assets and market microstructures; such examples are limit order books, trading volume, volatility surfaces and, as the focus of this paper, asset returns [5, 12, 20]. We refer to synthetic generated data for financial asset returns time series and its data generator as simply synthetic market data and market generator, respectively, from now on. There are generally two primary reasons [12] why academic researchers and practitioners are interested in generating synthetic market data:

- (1) **Forecasting and Prediction:** Synthetic market data can generate future market paths to used for training and evaluation of forecasting models, enabling researchers to test their algorithms on a wide range of scenarios. With recent advances in deep learning, generative models like GANs and VAEs, particularly their conditional variants, gives more desirable results as they can learn conditional distributions of financial time series data as opposed to point estimators [12].
- (2) **Simulating Market Scenarios:** The ability to generate synthetic market data allows researchers and practitioners to simulate various market scenarios alternatively from historical data, including extreme events and rare occurrences. This is particularly valuable for stress testing trading strategies, risk management models, and financial algorithms under different market conditions. It comes to no surprise as other domains like computer vision and natural language processing have seen similar trends in the use of synthetic data for training and testing models [16, 23]. However, financial market data is inherently more challenging due to its non-stationary, time-dependent nature, on top of the fact that there is only "one realization" of the market to which the market generator wishes to learn the distribution from. Market generators that captures historical data distributions can

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnn>

act as data augmentation to improve backtesting robustness and reduce overfitting to historical data [12].

We refer to Horvath et al. [12] for a more comprehensive historical overview of market generators and their applications in financial research and practice.

1.2 Limitations in the market generator Literature

Among other things, we observed that there is currently no universal or generally accepted approach to evaluating synthetic time series; this issue extends beyond sythetic market generator to generative frameworks like GANs as a whole [23] and time series generation [20]. Many evaluation measures are insufficiently defined and lack public implementations, making reuse and reproduction troublesome and error-prone [20]. This presents a challenge unique to the generation task compared to areas like time series forecasting or classification [20]. Hence, future research would immensely benefit from a widely accepted, reasonably sized set of qualified measures for the central evaluation criteria. Here we outline observations on the current limitations in market generator research and evaluation practices:

- [L1] **Inconsistent Evaluation Metrics:** Different studies employ varying metrics, hindering direct comparisons between models. For instance, some focus on statistical similarity, while others emphasize utility in downstream tasks [20].
- [L2] **Lack of Standardized Datasets and Preprocessing:** The absence of common benchmark datasets leads to evaluations on disparate data, making it challenging to assess model performance uniformly [20] in terms of both asset classes, data granularity, and data length. Furthermore, preprocessing steps (e.g., normalization, windowing) vary widely, affecting results and complicating comparisons.
- [L3] **Narrow Scope of Evaluation:** Many evaluations concentrate on a narrow set of criteria, such as marginal distributions, neglecting other important aspects like temporal dependencies, generation diversity, and downstream tasks [12, 20].
- [L4] **Reproducibility Issues:** The lack of open-source implementations and detailed evaluation protocols impedes reproducibility and validation of results across different studies [17, 20]. It is important to note that most model papers did not provide code for their evaluations.

1.3 Our Contributions

In this paper, we developed a unified market generator for historical financial time series on equity returns by adopting the time series evaluation taxonomy outlined from Stenger et al. [20], improve the time series generation framework from Ang et al. [3], and extending the utility evaluation frameworks implimented by Boursin et al. and outlined by Fecamp et al. [6, 9] (from option pricing deep hedging to portfolio hedging and alpha generation). We also investigate the various potential trade-offs between different evaluation metrics and categories of metrics. Our key contributions include:

- [C1] **Consolidated Evaluation Framework:** In effort to address [L1], we systematically review and consolidate evaluation

methods from leading papers (models and surveys) in market generators [3, 6, 20] and create a comprehensive assessment toolkit based on established practices.

- [C2] **Unified Statistical and Utility Measures:** For the first time, we integrate both statistical fidelity metrics and practical utility measures in a single benchmark, providing a more complete evaluation of synthetic data quality.
- [C3] **Outline Benchmark Framework:** We deliver an open-source benchmark framework to share with the research community, aiming to establish cohesion and standard for future market generator research in evaluation and comparison.

1.4 Research Questions

Our benchmark aims to address the following research questions:

- [Q1] **How do different sequence lengths during training affect market generator performance?** We investigate the impact of varying training sequence lengths on the performance of deep learning market generators as an extension of existing work like Allen et al. [2]. The selected training sequence lengths are described in §4.2.3.
- [Q2] **Is there a trade-off between certain metrics or categories of metrics?** In an effort to incorporate an ensemble of evaluation metrics, we are interested in understanding potential trade-offs between these metrics and how they impact the overall performance synthetic market data [3]. Particularly, we are interested in a market generator's ability to balance fidelity and diversity as highlighted in recent studies [16]. More importantly, we pay special care to the statistical quality of a market generator versus its utility when augmenting downstream task performance.
- [Q3] **Whether new deep learning models actually outperform classical parametric models in downstream tasks?** Most existing literature focuses on deep learning-based methods tend to dominate the field and claims their superiority in capturing complex market data distributions [5, 12, 21, 22]. We aim to provide a comprehensive comparison between deep learning-based approaches and traditional methods, focusing on their respective strengths and weaknesses in generating high-quality market data and its utility in downstream tasks.

2 Preliminaries

2.1 Problem Definition

Let us formally define the markt generation problem. Given a financial time series of asset prices movements $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_K)^T \in \mathbb{R}^{T \times K}$ with K individual securites spanning time T . Let $p(\mathbf{x}_1, \dots, \mathbf{x}_K)$ denote the real distribution of the given time series $\mathbf{X}$. The objective of the market generator is to synthesize a time series $\hat{\mathbf{X}} = (\hat{\mathbf{x}}_1, \dots, \hat{\mathbf{x}}_n)^T$ such that its distribution $q(\hat{\mathbf{x}}_1, \dots, \hat{\mathbf{x}}_n)$ approximates $p(\mathbf{x}_1, \dots, \mathbf{x}_K)$, while preserving the key properties exhibited in the real financial time series, such as autocorrelation structure, volatility clustering, and distributional moments.

2.2 Scope of Project

2.2.1 Evaluated Methods. Our benchmark encompasses a diverse range of models from traditional parametric approaches to modern deep learning architectures. We selected models satisfying one of three main criteria: (1) proven industry adoption, (2) research community acceptance, and (3) recent methodological innovations and interest.

Our evaluation includes classical parametric models, which rely on predefined statistical distributions and assumptions about the underlying data generation process. These models have been extensively used in financial institutions for their interpretability and theoretical foundations.

We also evaluate modern deep learning approaches, particularly deep learning-based models that learn the data distribution directly from observations without assuming a specific form. These include generative adversarial networks, and variational autoencoders, representing the cutting edge in synthetic market data generation.

An overview of the classes of market generator evaluate is outlined in §3.

2.2.2 Datasets. The market generators' objective is to generate realistic and diverse synthetic time series from daily financial asset returns, spanning across 10 years and collected from Yahoo Finance. In the interest of evaluating cross-asset distributive properties, we have select a range of financial equities, including 25 US large-cap stocks from the following sectors. Furthermore, we have included 5 of the most popular US index-tracking ETFs.

2.2.3 Scope of Evaluation Taxonomical Criteria. Our evaluation framework follows the taxonomy structure proposed by Stenger et al. [20], incorporating various evaluation measures from different papers in the field.

We systematically integrate evaluation metrics from multiple sources while maintaining this structured taxonomical approach, ensuring comprehensive coverage of all critical aspects of evaluation and setting a standard for future research in market generators as a whole. A full description of the evaluation suite are outlined in §5.1, 6.

3 Overview of Market Generator Methods

We categorize synthetic financial time series generation methods into two main families: classical stochastic (parametric) models, which rely on explicit assumptions about the data-generating process, and modern deep learning (deep learning) models, which learn complex dependencies directly from data without predefined functional forms. The following subsections provide a detailed overview of the methods included in our benchmark.

3.1 Classical Stochastic (Parametric) Methods

Parametric models assume a specific functional form for the data generating process, characterised by a finite set of parameters. These models are interpretable and analytically tractable, allowing for closed-form solutions in many cases. However, they may struggle to capture complex stylised facts or high-dimensional dependencies beyond their structural assumptions. Below we outline several widely-used parametric models in financial markets.

Table 1: Common notation and variables.

Symbol	Description
S_t	Asset price
$X_t = \ln S_t$	Log-price
$r_t = \ln(S_t/S_{t-1})$	Log-return
μ	Drift
σ	Volatility
W_t	Standard Brownian motion
Δt	Discrete time step

[A1] Geometric Brownian Motion (GBM). The price process follows a continuous-time stochastic process with constant drift and volatility. Model-specific parameters are μ and σ .

$$dS_t = \mu S_t dt + \sigma S_t dW_t. \quad (1)$$

[A2] Ornstein–Uhlenbeck (OU). A mean-reverting Gaussian process for log-prices or spreads. Model-specific parameters are $\theta > 0$ (reversion rate), μ (long-term mean), and σ (volatility).

$$dX_t = \theta(\mu - X_t) dt + \sigma dW_t. \quad (2)$$

[A3] Merton Jump Diffusion (MJD). Extends the GBM equation (1) by incorporating normally-distributed jumps. Model-specific parameters are λ (jump intensity), μ_j and σ_j (mean and standard deviation of jump sizes).

$$\frac{dS_t}{S_t} = \mu dt + \sigma dW_t + J dN_t, \quad (3)$$

where $J \sim \mathcal{N}(\mu_j, \sigma_j^2)$, $N_t \sim \text{Poisson}(\lambda t)$.

[A4] Double-Exponential Jump Diffusion (DEJD). Another jump-diffusion model with asymmetric double-exponential jump sizes modifying equation (1). Model-specific parameters are p (probability of upward jump) and η_1, η_2 (exponential parameters for upward/downward jumps).

$$\frac{dS_t}{S_t} = \mu dt + \sigma dW_t + Y dN_t, \quad (4)$$

$$Y = \begin{cases} \text{Exp}(\eta_1), & \text{with probability } p, \\ -\text{Exp}(\eta_2), & \text{with probability } 1 - p. \end{cases}$$

[A5] GARCH(1,1). A discrete-time conditional heteroskedastic model for returns. Model-specific parameters are $\omega > 0$, $\alpha \geq 0$, $\beta \geq 0$ (GARCH coefficients) and $\mathcal{D}$ (innovation distribution, usually standard Normal or Student-t). Captures volatility clustering via time-varying conditional variance.

$$r_t = \sigma_t z_t, \quad z_t \sim \mathcal{D}, \quad (5)$$

$$\sigma_t^2 = \omega + \alpha r_{t-1}^2 + \beta \sigma_{t-1}^2.$$

3.2 Deep Learning Methods

Deep learning models, in this context, refer to generative (often implicit) models that do not assume a fixed parametric form for the underlying data-generating process. Instead, they learn latent structures directly from data through flexible architectures such as neural networks, enabling the modeling of highly nonlinear,

high-dimensional, and multi-modal dependencies (e.g., multivariate financial time series with interactions across assets, time horizons, and features). While these models trade off interpretability and analytical tractability, they offer substantially greater expressive power and flexibility. This expressiveness, however, comes at the cost of increased data requirements, more intensive hyperparameter tuning, and higher computational demands.

We categorize these approaches into two major families of deep generative models, considered in this paper: Generative Adversarial Networks (GANs) and Variational Autoencoders (VAEs).

3.2.1 Variational Autoencoders (VAEs). VAEs [13, 19] are latent-variable generative models that posit a probabilistic encoder $q_\phi(z | x)$ and decoder $p_\theta(x | z)$. One maximizes the evidence lower bound (ELBO):

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)] - \text{KL}(q_\phi(z | x) \| p(z)).$$

Often one uses a standard Gaussian prior $p(z) = \mathcal{N}(0, I)$. The encoder-decoder structure allows sampling by first drawing $z \sim p(z)$, then generating $x \sim p_\theta(x | z)$.

In financial time-series generation, specialized versions such as the Time-Causal VAE (TC-VAE) have been proposed to enforce causality in the encoding/decoding of temporal data. For instance, Acciaio et al. (2024) propose TC-VAE, which ensures that generated paths respect temporal causality and show that the model reproduces stylized facts (e.g., heavy tails, volatility clustering) on real markets [1]. Because VAEs provide a principled latent representation, they are useful in finance to model underlying latent drivers of asset dynamics and to sample coherent trajectories.

[A6] TimeVAE. A time-series specific VAE variant that combines causal temporal encoders (e.g., causal convolutions or RNNs) with an autoregressive decoder to preserve path coherence. Training typically augments the ELBO with prediction or stepwise reconstruction losses to improve short- and long-range dependencies; TimeVAE is used in our benchmark as a dedicated VAE baseline for temporal data [8].

3.2.2 Generative Adversarial Networks (GANs). GANs models approach generation as a two-player game between a generator G and a discriminator D [10]. The discriminator is trained to maximize the probability of correctly classifying real data and generated data, with loss

$$L_D = -\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] - \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))].$$

The generator aims to fool the discriminator and is trained to minimize

$$L_G = -\mathbb{E}_{z \sim p(z)} [\log D(G(z))].$$

Many GAN variants, such as Wasserstein GAN or Least Squares GAN, modify these loss functions to promote more stable training and address issues like mode collapse [10].

In financial data synthesis, GANs have been widely used to generate realistic synthetic time series. For example, GAN-based financial data generation models (often using WGAN or improvements) show that one can improve the authenticity and predictive ability of generated financial statements or time series [18]. Another example is VRNNGAN, which uses a recurrent VAE as the generator and a recurrent discriminator to capture temporal dependencies in synthetic sequence generation [14]. GANs are relevant

in finance because they can capture complex joint distributions and dependencies in multivariate time-series without requiring explicit likelihood models.

[A7] QuantGAN. A GAN variant tailored for financial applications that incorporates quantile-aware objectives or conditional quantile matching and temporal architectures (e.g., temporal convnets or transformers). QuantGANs aim to better capture tail behaviour and risk-sensitive statistics (e.g., VaR/CVaR) important for downstream financial tasks [24].

3.3 Non-parametric

Non-parametric models are generator classes that does not rely on probability distribution assumptions or, in the case of Bayesian non-parametric models, does not require a specified model parameters. Among the simplest but most effective of the non-parametric models are the variations of time series block bootstrap that, in general, sample blocks of time data to preserve temporal dependencies. 3.2.

[A8] Block Bootstrap (Resampling Method). A resampling technique that preserves short-range dependence by sampling contiguous blocks of returns. Although not parametric, it is included here for completeness and baseline comparison.

4 Market Generator Framework

It is important to have standardized generation framework as part of our efforts to create a unified benchmark [C3]; this section outlines the details of the dataset used, preprocessing, and generation framework used by StonkBench for models to train on historical market data.

4.1 Dataset Specifications

The dataset contains daily close prices and trading volume for 20 U.S. large-cap stocks and 5 large index-tracking ETFs. The stock universe spans technology, financials, healthcare, consumer discretionary, industrials, communication services, consumer staples, energy, utilities, real estate, and materials. The ETF universe consists of broad-market and sector-focused funds. The dataset spans 15 years, from start of 2010 to the start of 2025 from Yahoo Finance.

For the generation framework, we focus on daily close-price time series. We represent the real dataset as $D := \{x_i\}_{i=1}^T = (x_{i,1}, x_{i,2}, \dots, x_{i,K})^T$.

4.2 Data preprocessing

4.2.1 Dataset Split and Transformation. Given our stock price dataset, we perform a chronological ten-two-three year partition to a training from 2010 to 2025, validation and testing dataset $D = D^{\text{train}}, D^{\text{val}}, D^{\text{test}}$, denoting concatenation. This deviates from the training scheme proposed by TSGBench [3]; we wish to hold out our testing time-horizon from the training set completely to evaluate model generalization capabilities.

4.2.2 Log Return Transformation. We then transform the price time series for both sets into a log return time series, which is a common practice in financial modeling to stabilize variance and make the data more suitable for analysis. The return r_t at time t is

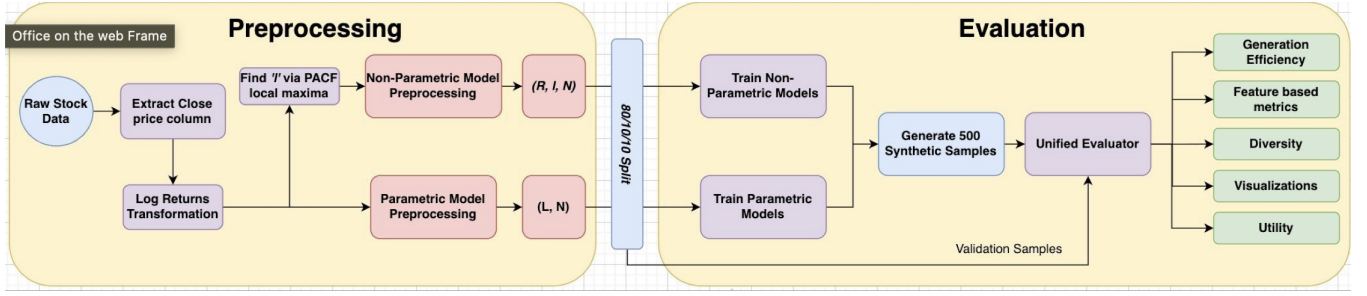


Figure 1: Stonk Bench system architecture overview.

Table 2: Asset classification used in StonkBench.

Ticker	Name	Stock/ETF	Industry/Exposure
AAPL	Apple	Stock	Technology
MSFT	Microsoft	Stock	Technology
NVDA	NVIDIA	Stock	Technology
AVGO	Broadcom	Stock	Technology
JPM	JPMorgan Chase	Stock	Financials
LLY	Eli Lilly	Stock	Healthcare
UNH	UnitedHealth Group	Stock	Healthcare
AMZN	Amazon	Stock	Consumer Disc.
TSLA	Tesla	Stock	Consumer Disc.
CAT	Caterpillar	Stock	Industrials
UNP	Union Pacific	Stock	Industrials
META	Meta Platforms	Stock	Communication
NFLX	Netflix	Stock	Communication
PG	Procter & Gamble	Stock	Consumer Staples
COST	Costco Wholesale	Stock	Consumer Staples
XOM	Exxon Mobil	Stock	Energy
CVX	Chevron	Stock	Energy
NEE	NextEra Energy	Stock	Utilities
PLD	Prologis	Stock	Real Estate
LIN	Linde	Stock	Materials
SPY	S&P 500	ETF	S&P 500
QQQ	Nasdaq-100	ETF	Nasdaq-100
XLF	Finance	ETF	U.S. Financials
IWM	Russell 2000	ETF	Small Caps
XLV	Health Care	ETF	U.S. Healthcare

calculated as:

$$r_t := \log \left(\frac{x_t}{x_{t-1}} \right) = \log(x_t) - \log(x_{t-1}), \quad (6)$$

where x_t is the daily closing price at day t . From now, on we refer to D as the log return time series dataset.

4.2.3 Sliding Window Transformation. For D^{train} , given a pre-determined window length $L \in \mathbb{N}$, we applying a sliding window operation W_L over the time series as follows.

$$W_L : \{x_i\}_{i=1}^T \mapsto \{(x_i, x_{i+1}, \dots, x_{i+L}) : 1 \leq i \leq T - L\} \quad (7)$$

For the purpose of research, we experiment our generation training with window lengths $L \in \{21, 63, 126, 252\}$, representing general

the number of trading days in a month, a quarter, half a year and a year, respectively ¹.

4.3 Market Generator Training and Generation

We provide a standard framework for training market generators over market data. Given a market generator $G_j, j \in \mathcal{J}$, where $\mathcal{J}$ denotes the index set of market generators to be trained and tested, we train the model on the log return training dataset with the objective to learn and generate sample k of synthetic log return time series $D_{G_j}^{(n)} \sim G_j(D^{train})$ that approximates the real log return time series D^{train} . For this paper we generate $N = 1000$ samples from the market generators and evaluate the generator based on the samples, denoted as $\mathcal{D}_{G_j} = \{D_{G_j}^{(n)}\}_{n=1}^N$.

4.3.1 Non-deep Learning Model Generation. For the context of the paper, the parametric and bootstrap market generators considered can train on the full time series D^{train} . For a parametric generator G_j , it learns a set of parameters $\hat{\theta}$ from D^{train} and generate $D_{G_j}^{(n)} \sim G_j(\hat{\theta}|D^{train})$ such that $D_{G_j}^{(n)}$ and D^{train} span over the same time length T .

For the block-bootstrap generator G_j , it applies a random-sampling operation over the entire training data D^{train} defined by the model to sample $D_{G_j}^{(n)} \sim G_j(D^{train})$ such that $D_{G_j}^{(n)}$ and D^{train} span over the same time length T .

4.3.2 Deep Learning Model Generation. The deep learning frameworks considered in this paper (i.e. GAN, VAE, DDPM, signature-based) requires to be trained on the sliding window transformation $W_L D^{train}$ (see 4.2) such that these model learn to generate a window of length L from $p(W_L D^{train})$. We propose, for a given deep learning model G_j that samples a window $\tilde{x}_{G_j}^{(L)} \sim G_j(\tilde{x}^{(L)}|W_L D^{train})$, to sample $(T\%L) + 1$ samples, where $\%$ denotes integer division, concatenate them sequentially, and trim out the right tail of the time series such that the resulting generated time series span the same time span T as the trained time series; we repeat this process

¹TSGBench determined the window length for deep learning training and generation depending on the dataset by finding the maximum auto-correlation of that time series dataset [3]

for $N = 1000$ times to obtain $\mathcal{D}_{G_j}$. Formally:

$$\left\{ \tilde{x}_m : \tilde{x}_m = \left(\tilde{x}_{G_j}^{(L)} \right)_m \right\}_{m=1}^{(T\%L)+1} \quad \tilde{x}_m \sim G_j(\tilde{x}^{(L)} | W_L D^{train}) \quad (8)$$

$$\tilde{D}_{G_j}^{(n)} := \tilde{x}_1, \dots, \tilde{x}_{(T\%L)+1} = (\mathbf{x}_1, \dots, \mathbf{x}_{T-1}, \mathbf{x}_T, \dots, \mathbf{x}_{T+L \bmod T}) \quad (9)$$

$$D_{G_j}^{(n)} = (\mathbf{x}_1, \dots, \mathbf{x}_{T-1}, \mathbf{x}_T) \quad (10)$$

$$\mathcal{D}_{G_j} := \left\{ D_{G_j}^{(n)} \right\}_{n=1}^N \quad (11)$$

5 Statistical Evaluation Measures

The statistical metrics considered in this paper are used to evaluate either the realism of synthetic market data to its trained history or generative novelty in market scenarios. The following subsections describes the metrics considered and the evaluation framework applied.

5.1 Statistical Measures

As described in [L1], our contribution [C1] is based on a well-established taxonomy framework [20] while introducing our industry-specific stylized facts to holistically evaluate the characteristics of synthetic market data. We divide our statistical metrics

5.1.1 Fidelity Measures. These metrics quantify how well synthetic data captures key marginal and second-order properties of real data. We employ the following feature-based metrics:

[M1] Marginal Distribution Difference (MDD): To measure the distance between empirical marginal distributions of real versus synthetic data, we use the 1-Wasserstein distance (a.k.a. Earth Mover's distance). This was chosen over the popular Kullback-Leibler divergence or the derivatives Jensen-Shannon divergence due to its better numerical stability and ability to capture distributional differences even when supports do not overlap [4].

$$\text{MDD} = W_1(\hat{F}_{\text{real}}, \hat{F}_{\text{synth}}),$$

$$W_1(F, G) = \int_0^1 |F^{-1}(u) - G^{-1}(u)| du,$$

where $\hat{F}_{\text{real}}$ and $\hat{F}_{\text{synth}}$ are empirical CDFs.

The following four metrics capture differences in the first four statistical (central) moments between real and synthetic data.

[M2] Mean Difference (MD):

$$\text{MD} = |\mu_{\text{real}} - \mu_{\text{synth}}|.$$

[M3] Standard Deviation Difference (SDD):

$$\text{SDD} = |\sigma_{\text{real}} - \sigma_{\text{synth}}|.$$

[M4] Skewness Difference (SD):

$$\text{SD} = |\gamma(r_{\text{real}}) - \gamma(r_{\text{synth}})|,$$

$$\gamma(r) = \frac{\mathbb{E}[(r - \mu)^3]}{(\sigma)^3}.$$

[M5] Kurtosis Difference (KD):

$$\text{KD} = |\kappa^{\text{ex}}(r_{\text{real}}) - \kappa^{\text{ex}}(r_{\text{synth}})|,$$

$$\kappa^{\text{ex}}(r) = \frac{\mathbb{E}[(r - \mu)^4]}{(\sigma)^4} - 3.$$

5.1.2 Visualization Methods. Visual tools complement numeric metrics for face-validity and communication [3].

[M6] t-SNE: 2D embeddings of window-level features for real versus synthetic overlap and cluster structure.

[M7] Distribution: Histograms of returns to visualize empirical distribution.

5.1.3 Diversity Measures. We are also interested if the SDGFTS can generate data not observed but follow the same distribution as the real data. Therefore, we include the following intra-class distance metrics. The inclusion of these metrics are chose to ensure that mode collapse is prevented.

[M8] Intra-Class Euclidean Distance (ICD-ED):

$$\text{ICD-ED} = \frac{2}{R(R-1)} \sum_{1 \leq a < b \leq R} \|\mathbf{r}_a - \mathbf{r}_b\|_2,$$

where $\mathbf{r}_a \in \mathbb{R}^L$ is the a -th sample.

[M9] Intra-Class Dynamic Time Warping (ICD-DTW):

$$\text{ICD-DTW} = \frac{2}{R(R-1)} \sum_{1 \leq a < b \leq R} d_{\text{DTW}}(\mathbf{r}_a, \mathbf{r}_b).$$

5.1.4 Efficiency Measures. Generation efficiency is crucial for practical applications, especially in high-frequency trading or real-time risk management, where rapid data synthesis is required. Thus, we include the following efficiency metric [20].

[M10] Generation Time: Wall-clock time to generate S samples:

$$T_{\text{gen}} = t_1 - t_0 \quad (\text{seconds for } 1,000 \text{ samples}).$$

5.1.5 Stylized Facts Measures. Canonical stylized facts:

[M11] Autocorrelation Difference (ACD): Absolute difference in lag-1 autocorrelation:

$$\rho(1) = \frac{\sum_{t=2}^L (r_t - \bar{r})(r_{t-1} - \bar{r})}{\sum_{t=1}^L (r_t - \bar{r})^2},$$

$$\text{ACD} = |\rho_{\text{real}}(1) - \rho_{\text{synth}}(1)|.$$

[M12] Volatility Clustering (VC): Lag-1 autocorrelation of squared or absolute returns measures volatility clustering, a stylized fact where large price movements tend to cluster in time [7]:

$$\rho_{r^2}(1) = \text{Corr}((r_t)^2, (r_{t-1})^2),$$

$$\rho_{|r|}(1) = \text{Corr}(|r_t|, |r_{t-1}|).$$

[M13] Long Memory / Slow Decay (LMSD): To quantify long-range dependence in volatility, we follow Cont (2001) [7] and study the autocorrelation of absolute (or squared) returns:

$$C_\alpha(\tau) = \text{corr}(|r_{t+\tau}|^\alpha, |r_t|^\alpha), \quad \alpha \in \{1, 2\}.$$

Empirically, this correlation decays slowly according to a power law,

$$C_\alpha(\tau) \sim \frac{A}{\tau^\beta}, \quad \beta \in [0.2, 0.4],$$

indicating persistent volatility fluctuations (long memory). For evaluation, we estimate the decay exponent by a log-log linear fit over lags $\tau \in \{1, \dots, \tau_{\text{max}}\}$ where $C_\alpha(\tau) > 0$:

$$\hat{\beta} = -\text{slope}(\log C_\alpha(\tau) \text{ vs. } \log \tau).$$

Our metric is the absolute difference between decay exponents estimated on real and synthetic data:

$$\text{LMSD} = |\widehat{\beta}_{\text{real}} - \widehat{\beta}_{\text{synth}}|.$$

Unless otherwise stated, we set $\alpha = 1$ (absolute returns) and report $\widehat{\beta}$ together with the goodness-of-fit of the linear regression.

5.2 Statistical Evaluation Framework

We compare the statistical measures outlined above between the synthetic market data to the trained historical dataset. Consider a market generator $G_j, j \in \mathcal{J}$ trained on the dataset D^{train} and its generated data $\mathcal{D}_{G_j}$ as specified in 4.3, we can apply an evaluation metric E , as mentioned in 5.1, defined an measurable function $\mathbb{R}^{T \times K} \rightarrow \mathbb{R}$.

6 Utility Evaluation of Downstream Tasks

Most market generator research, particularly in this sector, are primarily interested in the statistical similarities while overlook experimenting on downstream effects of synthetic market data in practical settings [12, 20]. While many works consider downstream scenarios such as option delta (deep) hedging [6, 22], core time series generation benchmark such as TSGBench fail to incorporate such experiments due to the various industries that Ang et. al aimed to benchmark.

We propose three (3) experiments to demonstrate the downstream effects of synthetic market data through a train-synthetic-test-real (TSTR) framework to replicate realistic application of synthetic market data to financial institutions and market participants will use [12, 20]. In addition to the utility of option deep hedging, a popular utility benchmark among market generator papers [6, 22], for the first time in the literature, we introduce the application of risk management/portfolio hedging from the perspective of an equity market maker, and portfolio construction/alpha generation from the perspective of a hedge fund manager or proprietary trader.

6.1 Overview of Utility Evaluation

The general framework of utility evaluation is structured the same across our three downstream tasks, centering around the train-synthetic-test-real (TSTR) evaluation framework [15] with a rolling-window validation for hyperparameter tuning, adapted for financial time series generation. We begin with our real market dataset part partition as specified in 4.2 —along with a specific learning task $\mathcal{T}$ and a performance score S to evaluate task effectiveness. We consider an algorithm A (e.g. a 5-layer perceptron) that takes as input the dataset D and task $\mathcal{T}$.

In the context of market generator evaluation, we consider a market generator G_{TS} trained on D^{train} and validated with D_r^{validate} to generate synthetic FTS data $\mathcal{D}_g$.

Now we can train an algorithm A on the datasets D_r^{train} and $\mathcal{D}_g$ respectively, resulting in two models M_r and M_g .

Finally, we evaluate both models on the real test set D_r^{test} to obtain the corresponding performance scores s_r and s_g .

The goal of the (deep) hedger portfolio evaluation is to evaluate the effectiveness of the model M_{TS} with downstream tasks (hedging) by comparing the scores s_r and s_g . The model M_{TS} is

considered effective if the the results yielded similar scores, i.e. $s_r \approx s_g$ —preferably having higher performance with results of $s_g > s_r$ —indicating that the synthetic data generated by M_{TS} is useful for training a (deep) hedger.

6.1.1 Augmented Testing. To better evaluate the performance of (deep) hedgers trained on synthetic data, we train our (deep) hedger A on a new dataset D_{aug} that combines both real training data D_r^{train} and synthetic data $\mathcal{D}_g$. In other words, we train model M_g on D_{aug} where $D_{\text{aug}} := D_r^{\text{train}} \cup \mathcal{D}_g$. Then we proceed to evaluate scores between models M_g and M_r on the real test set, where M_r is still trained on only real data D_r^{train} . This extension, inspired by Stenger et al. [20], allows us to assess whether synthetic data provides additional value when combined with real training data.

6.2 Option Delta Hedging

Option hedging is employed by options market making trading desks to minimize the downside counter-party risk when a call expires or a put is exercised. By the two fundamental theorem of asset pricing, the option contract can be priced and replicated using the underlying security and the money market. For this utility evaluation, we take a perspective of an options market maker selling a European call option of a cash equity at strike K and maturity T .

6.3 Portfolio Hedging

Portfolio hedging is a common practice among equity market makers to mitigate the risk of adverse price movements in their inventory. In this utility evaluation, we simulate a market maker's inventory management strategy using synthetic market data generated by the market generator G . The market maker aims to maintain a balanced portfolio by dynamically adjusting their positions in response to market fluctuations.

Classical market-making models assume that dealers can reduce inventory risk through an external hedging venue once inventory becomes sufficiently large [11]. To mitigate aggregate market and sector risk, we assume the market maker can partially hedge accumulated stock inventory using broad-market and sector ETFs. This reflects common industry practice of using highly liquid index-tracking instruments as temporary hedges before inventory is fully unwound. While classical market-making models generally represent hedging via a generic external market rather than a specific instrument, our implementation instantiates this external hedge using liquid U.S. ETFs (see table 2), which serve as liquid proxies for market and sector exposures.

6.3.1 Portfolio Hedging Definition. For a time horizon T and a universe of K assets, we define the market maker's inventory at time t as $I_t := (I_{t,1}, \dots, I_{t,K}) \in \mathbb{R}^K$. Inventory positions are modeled as continuous exposures rather than integer share counts, consistent with the portfolio-level nature of the daily hedging problem.

Given a hedging model M trained on market data and a market generator P that produces synthetic price trajectories, the portfolio hedging task is to find a set of hedge positions $H = \{h_{t,j}\}_{t=1,j=1}^{T,J}$ in $J = 5$ liquid ETFs that minimize the notional exposure of the inventory portfolio netted against the ETF hedge portfolio as follows. Formally, the objective is to solve the following optimization

problem:

$$\min_H \sum_{t=1}^{T-1} \left| \sum_{i=1}^K x_{t+1,i} I_{t,i} - \sum_{j=1}^J \tilde{x}_{t+1,j} h_{t,j} \right| \quad (12)$$

where $x_{t,i}$ and $\tilde{x}_{t,j}$ are the daily log returns of the inventory and ETF per share, respectively. The hedging model M learns to dynamically adjust the hedge positions across the five ETFs to offset inventory risk and maintain a market-neutral stance.

6.3.2 Inventory Simulation. Since proprietary inventory records are unavailable, we model inventory as a stationary first-order autoregressive process. This choice is motivated by the inventory-risk framework of Guéant et al., where optimal quoting strategies continuously adjust bid and ask quotes to reduce inventory exposure toward a target level [11]. Rather than explicitly simulating the market-making strategy and order arrivals, we adopt a reduced-form approximation that preserves the primary stylized fact of dealer inventories: mean reversion toward a risk-neutral inventory level.

We consider a mean-reverting Ornstein–Uhlenbeck process for each asset i :

$$I_t = \alpha I_{t-1} + \sqrt{1 - \alpha^2} \Sigma^{1/2} \epsilon_t \quad (13)$$

where $\alpha \in (0, 1)$ is the “persistence” parameter of the inventory process, $\Sigma^{1/2}$ is the Cholesky decomposition of the sample covariance matrix of the inventory shocks, and $\epsilon_t \sim \mathcal{N}_K(0, I)$ is a standard K -dimensional Gaussian white noise.

6.4 Alpha Generation

References

- [1] Beatrice Acciaio, Stephan Eckstein, and Songyan Hou. 2024. Time-Causal VAE: Robust Financial Time Series Generator. arXiv:2411.02947 [cs.LG] <https://arxiv.org/abs/2411.02947>
- [2] David E. Allen, Leonard Mushunje, and Shelton Peiris. 2025. GANs and synthetic financial data: calculating VaR*. *Applied Economics* 57, 37 (2025), 5680–5695. arXiv:<https://doi.org/10.1080/00036846.2024.2365456> doi:10.1080/00036846.2024.2365456
- [3] Yihao Ang, Qiang Huang, Yifan Bao, Anthony K. H. Tung, and Zhiyong Huang. 2023. TSGBenchmark: Time Series Generation Benchmark. *Proc. VLDB Endow.* 17, 3 (2023), 305–318.
- [4] Martin Arjovsky, Soumith Chintala, and Léon Bottou. 2017. Wasserstein Generative Adversarial Networks. *Proceedings of the 34th International Conference on Machine Learning* 70 (2017), 214–223.
- [5] Samuel Assefa, Danial Dervovic, Mahmoud Mahfouz, Tucker Balch, Prashant Reddy, and Manuela Veloso. 2019. Generating synthetic data in finance: opportunities, challenges and pitfalls. In *NeurIPS 2019 Workshop on AI in Financial Services: Data, Fairness, Explainability, Trustworthiness, and Privacy*. Vancouver, Canada. <https://ssrn.com/abstract=3634235>
- [6] Nicolas Boursin, Carl Remlinger, Joseph Mikael, and Carol Anne Hargreaves. 2022. Deep Generators on Commodity Markets; application to Deep Hedging. arXiv:2205.13942 [q-fin.RM] <https://arxiv.org/abs/2205.13942>
- [7] R. Cont. 2001. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance* 1, 2 (2001), 223–236. arXiv:<https://doi.org/10.1080/713665670> doi:10.1080/713665670
- [8] Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. 2021. TimeVAE: A Variational Auto-Encoder for Multivariate Time Series Generation. arXiv:2111.08095 [cs.LG] <https://arxiv.org/abs/2111.08095>
- [9] Simon Fecamp, Joseph Mikael, and Xavier Warin. 2021. Deep learning for discrete-time hedging in incomplete markets. *The Journal of Computational Finance* (01 2021). doi:10.21314/JCF.2021.006
- [10] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative Adversarial Networks. arXiv:1406.2661 [stat.ML] <https://arxiv.org/abs/1406.2661>
- [11] Olivier Guéant, Charles-Albert Lehalle, and Joaquin Fernandez Tapia. 2011. Dealing with the Inventory Risk. A solution to the market making problem. arXiv:1105.3115 [q-fin.TR] <https://arxiv.org/abs/1105.3115>
- [12] Blanka Horvath, Jonathan Plenk, and Milena Vulić. 2025. Market Generators: A Paradigm Shift in Financial Modeling. In *Signature Methods in Finance*. Springer Nature Switzerland, Cham, 127–159. doi:10.1007/978-3-031-97239-3_4
- [13] Diederik P Kingma and Max Welling. 2022. Auto-Encoding Variational Bayes. arXiv:1312.6114 [stat.ML] <https://arxiv.org/abs/1312.6114>
- [14] J. Lee et al. 2022. VRNNGAN: A Recurrent VAE-GAN Framework for Synthetic Time-Series Generation. In *Conference*.
- [15] Mark Leznik, Patrick Michalsky, Peter Willis, Benjamin Schanzel, Per-Olov Östberg, and Jörg Domaschka. 2021. Multivariate Time Series Synthesis Using Generative Adversarial Networks. In *Proceedings of the ACM/SPEC International Conference on Performance Engineering (Virtual Event, France) (ICPE '21)*. Association for Computing Machinery, New York, NY, USA, 43–50. doi:10.1145/3427921.3450257
- [16] Chung I Lu and Julian Sester. 2025. Generative modelling of financial time series with structured noise and MMD-based signature learning. arXiv:2407.19848 [q-fin.MF] <https://arxiv.org/abs/2407.19848>
- [17] Vamsi K. Potluru, Daniel Borrajo, Andrea Coletta, Niccolò Dalmaso, Yousef El-Laham, Elizabeth Fons, Mohsen Ghassemi, Sriram Gopalakrishnan, Vikesh Gosai, Eleonora Kreacic, Ganapathy Mani, Saheed Obitayo, Deepak Paramanand, Natraj Raman, Mikhail Solonin, Srijan Sood, Svitlana Vyetenko, Haibei Zhu, Manuela Veloso, and Tucker Balch. 2024. Synthetic Data Applications in Finance. arXiv:2401.00081 [cs.LG] <https://arxiv.org/abs/2401.00081>
- [18] Feng Qi. 2025. GAN-Based Financial Data Generation and Prediction: Improving The Authenticity and Prediction Ability of Financial Statements. *Informatica* (2025).
- [19] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. 2014. Stochastic backpropagation and approximate inference in deep generative models. In *Proceedings of the 31st International Conference on International Conference on Machine Learning - Volume 32 (Beijing, China) (ICML '14)*. JMLR.org, II–1278–II–1286.
- [20] Michael Stenger, Robert Leppich, Ian Foster, Samuel Kounev, and André Bauer. 2024. Evaluation is key: a survey on evaluation measures for synthetic time series. *J Big Data* 11, 66 (May 2024). doi:10.1186/s40537-024-00924-7
- [21] Tomonori Takahashi and Takayuki Mizuno. 2024. Generation of synthetic financial time series by diffusion models. arXiv:2410.18897 [q-fin.CP] <https://arxiv.org/abs/2410.18897>
- [22] Yuki Tanaka, Ryuji Hashimoto, Takehiro Takayanagi, Zhe Piao, Yuri Murayama, and Kiyoshi Izumi. 2025. CoFinDiff: Controllable Financial Diffusion Model for Time Series Generation. arXiv:2503.04164 [q-fin.CP] <https://arxiv.org/abs/2503.04164>
- [23] Zhengwei Wang, Qi She, and Tomas E. Ward. 2020. Generative Adversarial Networks in Computer Vision: A Survey and Taxonomy. arXiv:1906.01529 [cs.LG] <https://arxiv.org/abs/1906.01529>
- [24] Magnus Wiese, Robert Knobloch, Ralf Korn, and Peter Kretschmer. 2020. Quant GANs: Deep Generation of Financial Time Series. *Quantitative Finance* 20, 9 (April 2020), 1419–1440. doi:10.1080/14697688.2020.1730426